How to verify v2 and check that the output is correct

There are four levels of checking. Each one ends with a clear pass/fail, so you never have to judge a waveform by eye to know whether the design works.

level	what	tool	time	pass looks like
1	each block on its own (14 unit testbenches)	Icarus or Vivado XSim	~3 min	`ALL n CHECKS PASSED` for every test
2	the whole controller as the ARM sees it, the whole FPGA-only board image, and that image through its pins only (also as a gate netlist)	Icarus or XSim	~10 min	`ALL 36 CHECKS PASSED`, `ALL 8 CHECKS PASSED`, `ALL 11 CHECKS PASSED`
3	waveforms, when you want to see *why*	GTKWave or the Vivado wave window	-	the pictures described in section 4
4	the real ZedBoard	Vivado + Vitis + PC + logic analyser	1-2 h	the results in sections 5 and 6

The slide-23 promises map onto these tests like this:

slide 23	test	pass criterion
serial engine: UART, SPI and I2C programs match the v1 engines bit for bit	`tb_se_programs`	same bit streams, same bus events, same bytes (69 checks)
routers: random traffic from all six nodes, no lost, repeated or reordered packets, no deadlock	`tb_noc_router`, `tb_noc_mesh`, `tb_noc_router_equiv`	scoreboard: every packet once, unchanged, in order per flow; long runs finish
crossbar: random switch requests, never mid-frame, never two drivers on one pin	`tb_xbar`	reference model checked every clock + "no switch mid-frame" property (1.2 million checks)
hub and DMA: every record at its address, the pointer wraps, interrupt after 32	`tb_hub`, `tb_dma_writer`, `tb_ssc2_system`	memory compared with a shadow copy; batch event exactly after record 32; 1 IRQ per 32 readings
on the board: flash ID 20 BA 19, temperature 0x0C80 = 25.00 C	`tb_ssc2_system`, `tb_zed2_standalone`, then the board	section 5 and 6

---

1. Unit tests (one block at a time)

Linux or WSL with Icarus Verilog (`sudo apt install iverilog`):

```
cd smart_serial_controller_v2
sh sim/run_unit.sh               # all of them
sh sim/run_unit.sh noc_mesh      # only tests whose name contains "noc_mesh"
```

Windows with Vivado only: `windows\1_simulate.bat` runs the most important ones in XSim. Any single test also runs from the Vivado Tcl console:

```
cd C:/ssc/smart_serial_controller_v2
set argv tb_xbar ; source vivado/run_sim.tcl
```

Each testbench is self-checking: it counts its checks and ends with one of

```
ALL n CHECKS PASSED
m OF n CHECKS FAILED        (and a "FAIL: ..." line for each failure)
```

`run_unit.sh` finishes with `UNIT TESTS: 14 passed`, or lists the tests that failed. The full log of each test is in `build/<test>.log`. Reference logs from a passing run are in `docs/sim_results/<test>.log`: compare yours with them (the lines printed by the testbench must match; XSim adds its own INFO lines around them).

What each unit test proves, and its last lines when everything is right:

test	block	what it proves	last lines (pass)
`tb_noc_pkt`	packet sender/receiver, time base	400 random packets through one link, flit kinds and padding, credit back-pressure (only 4 flits without credits), us/ms ticks	`A: 399 packets, 3548 flits, 399 good / ALL 19 CHECKS PASSED`
`tb_noc_router`	one router	routing to every direction, priority beats round-robin, fairness, wormhole (packets never interleave), credits, latency per hop, 1 flit/clock streaming	`T1: one 200-flit packet LOCAL->EAST in 200 clocks = 1.000 flits/clock / ALL 1547 CHECKS PASSED`
`tb_noc_router_equiv`	router timing fix	the timing-optimised router and the simple reference router (`sim/models/noc_router_ref.v`) side by side at all six mesh positions, same random inputs (with and without credit flow control, broken packets, resets): every output equal at every clock (TIMING.md)	`(2,1): 50000 clocks, ... 0 mismatches / ALL 18 CHECKS PASSED`
`tb_noc_mesh`	2x3 mesh (+ a 4x4)	random all-to-all traffic with the packet helpers at every node: every packet exactly once, unchanged, in order per flow; hot-spot; no deadlock in long runs; all credits back	`A: 5742 packets ... / ALL 19 CHECKS PASSED`
`tb_xbar`	pin crossbar	a cycle-accurate reference model checked every clock against random traffic and random switch requests; never switches mid-frame; FLOAT exactly 1 clock; SW_CYCLES/SW_EDGE	`COVERAGE: ... / ALL 1217806 CHECKS PASSED`
`tb_se_engine`	serial engine	every instruction and JMP condition, shifts both ways, PUSH/PULL, side-set, delays, OD, CRC ("123456789" -> 0x29B1), fractional divider, EXEC, RESTART, and the slide-13 program at 115200 baud	`measured bit time 8680.0 ns (ideal 8680.6) / ALL 257 CHECKS PASSED`
`tb_se_programs`	the programs	UART, SPI and I2C programs against the v1 engines bit for bit, Manchester loop with CRC, OLED timing	`CRC-16 over the received bits: SE 6dfd, model 6dfd / ALL 69 CHECKS PASSED`
`tb_ni_se`	serial-engine NI	bytes to the right SM and bit position, RX words to packets, flush rules, stall, a stopped SM cannot block the network	`ALL 19 CHECKS PASSED`
`tb_ni_uart`	UART NI	slide-17 frame `01 9F 03` -> request; reply -> PC gets `20 BA 19`; RAW, ADDRESSED, time-outs, back-pressure, stall	`ALL 65 CHECKS PASSED`

test	block	what it proves	last lines (pass)
`tb_ni_spi`	SPI NI	flash READ ID through the real v1 SPI engine, one CS# window, long reads, bad requests (status 4)	`ALL 50 CHECKS PASSED`
`tb_ni_i2c`	I2C NI	TMP2 read through the real v1 I2C engine (repeated START), NACK (status 1), time-out (status 3), wlen > 16 (status 4)	`ALL 77 CHECKS PASSED`
`tb_hub`	sensor hub	the slide-18 task, periods and phases, RUN_NOW, task order, records field by field, wrong tags, time-outs, errors	`hub: 59 packets, 40 requests, 19 records ...` / `ALL 197 CHECKS PASSED`
`tb_dma_writer`	DMA writer	every record at BASE + 16 x index, ring wrap, ring full (overflow), batch after 32, time-out batch, AXI errors, random back-pressure, AXI protocol	`ALL 242 CHECKS PASSED`
`tb_study`	NoC study fabrics	bus, crossbar and mesh deliver every flit correctly at low and full load	`ALL 21 CHECKS PASSED`

`tb_dma_writer` prints 7 `PROTOCOL ERROR` lines from `axi_mem_selftest`: they are deliberate (the test proves the memory model's own checker works) and the line before them says so.

---

2. System tests (the whole design)

```
sh sim/run_system.sh             # both, about 10 minutes
sh sim/run_system.sh system      # tb_ssc2_system only (~3 min)
sh sim/run_system.sh standalone  # tb_zed2_standalone only (~7 min)
```

In XSim: `set argv tb_ssc2_system ; source vivado/run_sim.tcl`.

2.1 `tb_ssc2_system`: Way 2, exactly as the ARM sees it

`ssc2_axi_top` driven over AXI4-Lite (like `sw/ssc2_driver.c`), its DMA master writing into an AXI memory model (the DDR), its pins going through the board pads to models of the PmodUSBUART (PC terminal), PmodSF3, PmodTMP2 and a logic-analyser UART decoder on JD. One "millisecond" is 10 us in this test so 32 readings take milliseconds.

Expected output (abridged; every line must say `ok`):

```
[1] registers
    ok    ID (v1 block, version 2) = 0x53534302
    ok    VERSION = 0x20000
    ...
[2] slide 17, flow A: PC -> UART NI -> SPI flash -> PC
    ok    PC got 3 reply bytes
    ok    flash ID through the NoC = 20 BA 19 = 0x20ba19
[3] slides 18-20, flow B: hub -> I2C -> hub -> DMA -> DDR, 1 IRQ per 32
    ok    batch interrupt arrived
    measured: 0 APB accesses and 1 interrupt(s) for 32 readings
    ok    CPU made 0 bus accesses during the 32 readings = 0x0
    ok    exactly 1 interrupt for 32 readings = 0x1
    ok    32 records: ts every 200 us, source 3, len 2, seq 1..32, 0C 80 = 0x20
    first record: timestamp ... us, temperature 0x0C80 = 25.00 C
[4] flow A while flow B was running
    ok    flash ID during the hub traffic = 20 BA 19 = 0x20ba19
[5] slide 14: JD switches from the SPI engine to SM0 (UART program)
    ok    switch is waiting while the SPI frame runs
    ok    JD now driven by SM0 = 0x4
    ok    the SPI frame had finished before the hand-over
    SW_CYCLES[JD] = 39325 clocks (waited for the SPI frame)
    ok    slide-13 program sends 'A' 'B' on JD2 at 115200 baud
    switch time from an idle source: 2 clocks
[6] protocol conversion: PC UART -> NoC -> SM0 UART on JD
    ok    "Hi!" from the PC came out of SM0 on JD
[7] v1 path: CPU reads the TMP2 itself (slide 20, left side)
    ok    v1-style read gives 0x0C80 = 0xc80
    measured v1 style: 10 APB accesses and 2 interrupts for 1 reading
    ok    AXI memory model saw no protocol or address errors
ALL 36 CHECKS PASSED
```

What this proves: the register map, both data flows of slides 17-19, the DMA path into memory, the slide-14 crossbar demo, protocol conversion through the NoC, v1 compatibility, and the CPU-load numbers of slide 20 (v2: 0 accesses + 1/32 interrupt per reading; v1: 10 accesses + 2 interrupts per reading).

2.2 `tb_zed2_standalone`: Way 1, the complete FPGA-only image

`zed2_top_standalone` with no CPU: `ssc2_boot` sets everything up.

```
[1] boot sequencer               ok  set-up list done, loop running
[2] PC reads the flash ID        ok  PC got 20 BA 19
[3] hub records in the block RAM ok  4 records: source 3, len 2, seq 1..4, data 0C 80
                                 ok  timestamps 100 "ms" (5000 us) apart
                                 ok  ring is freed by the boot loop (RD_IDX follows WR_IDX, no
overflow)
[4] LEDs                         ok  LD0 = record count / 4 bit 0
[5] SW = 4: JD driven by SM0     ok  JD2 shows the temperature bytes 0C 80 as UART frames
[6] SW7 = 1: ADDRESSED frame     ok  PC got the temperature 0C 80
ALL 8 CHECKS PASSED
```

---

3. When a check fails

1. Read the `FAIL:` line; it names what was expected and what came out.
2. Open `build/<test>.log`: the lines before the failure show which part was running.
3. Look at the waveform (section 4) around the time printed in the FAIL line (`t=...` is in picoseconds in Icarus).
4. If you changed a `.se` program, run `sh tools/build_programs.sh` again: the testbenches load `programs/*.hex`.

2.3 `tb_zed2_gate`: the board image through its pins only (also gate level)

The same Way 1 image, but the testbench only uses the pins of `zed2_top_standalone` and real time (no internal names, no shortened milliseconds). That is what a synthesised or implemented netlist offers, so this one test also runs as **post-synthesis** and **post-implementation** (functional and timing) simulation in Vivado (`docs/VIVADO_GUI.md`). Switches: SW7 = 1 (ADDRESSED frames), SW2..0 = 2 (JD shows the SPI pins).

```
[1] flash ID through the NoC: 04 00 01 9F 03
    ok    PC received 3 bytes
    ok    they are 20 BA 19
[2] crossbar: JD shows a copy of the SPI pins (SW2..0 = 2)
    ok    JD1 (CS#) went low
    ok    JD4 (SCK) clocked at least 4 bytes
[3] temperature through the NoC: 05 4B 01 00 02
    ok    PC received 2 bytes
    ok    they are 0C 80 (25.00 C)
[4] probe copies on JD7..10
    ok    JD7 (UART TXD copy) moved
    ok    JD9 (I2C SCL copy) moved
[5] fixed outputs
    ok    JB7..10 (CS1#, RST#, WP#, HOLD#) high
    ok    OLED supplies off
    ok    LD7 (interrupt line) off
ALL 11 CHECKS PASSED
```

`sh sim/run_system.sh gate` runs it on the Verilog (about 20 s).

4. Looking at waveforms

Icarus: `sh sim/run_system.sh system vcd` writes `build/tb_ssc2_system.vcd` (the testbench level and the controller's ports). Open it with `gtkwave build/tb_ssc2_system.vcd`. For a unit test, add `$dumpfile("build/x.vcd"); $dumpvars(0, tb_name);` to its `initial` block.

Vivado: double-click `windows\0_create_project_gui.bat`, right-click the simulation set you want (e.g. `sim_system`) -> *Make Active*, click *Run Simulation > Run Behavioral Simulation*, then in the Tcl console type `log_wave -r /`, `restart` and `run all`. Add signals from the Scope window. Details, including post-synthesis and post-implementation simulation: `docs/VIVADO_GUI.md`.

What the important waveforms look like:

- **UART frame on JA2 / JD2 (slide 13):** idle high, start bit low 8.68 us, 8 data bits LSB first, stop bit high. 'A' = 0x41 = `0 1000 0010 1`.
- **SPI READ ID on JB:** CS# low for 4 bytes; MOSI `9F` then zeros; MISO `xx 20 BA 19`; SCK 1 MHz, mode 0 (MOSI changes while SCK is low).
- **I2C TMP2 read on JC3/JC4:** START, `96` ACK, `00` ACK, repeated START, `97` ACK, `0C` ACK, `80` NACK, STOP; SCL 100 kHz.
- **Flits in the network:** `dut.u_core.u_noc` -> `tx_valid` / `rx_valid` per node; a head flit's bits [28:26] are the destination node.
- **A crossbar switch:** `pad_oe` for the port drops to 0 for exactly one clock (FLOAT), then the new source drives; `ev_switch` pulses once.
- **A DMA burst:** `M_AXI_AWVALID` with `AWLEN = 3`, then 4 W beats, `WLAST` on the 4th, then `BVALID`.

---

5. On the board: Way 1 (FPGA only)

Hardware: PmodUSBUART on JA (top row), PmodSF3 on JB (top row, and its bottom row on JB7-10), PmodTMP2 on JC pins 3-6 / 9-12 (so SCL = JC3, SDA = JC4), logic analyser on JD (optional), all switches down.

After programming `zed2_top_standalone.bit` (and pressing BTNC once):

check	how	correct result
it runs	look at the LEDs	LD4 blinks once a second (heartbeat)
records are made and stored	LD0-LD3	count up in binary, LD0 changes every 0.4 s (records / 4, 10 records per second)
batches	LD5	toggles every 3.2 s (32 records)
no errors	LD6	stays off (on = a hub request failed: check the TMP2 and its pull-ups)
flash ID through the NoC	`python tools/pc_frame.py COMx 01 9F 03`	`reply: 20 BA 19`
read 16 flash bytes	`python tools/pc_frame.py COMx 04 03 00 00 00 10`	16 bytes (FF on an erased flash)
temperature through the NoC	SW7 up, then `python tools/pc_frame.py COMx 05 4B 01 00 02 --temp`	`reply: 0C 80` -> `temperature: 25.0000 C` (your room temperature)
crossbar switching	SW2..0 = 010 (2): JD shows the SPI lines; 100 (4): JD2 shows UART frames from SM0 (the temperature bytes every 100 ms)	the logic analyser decodes SPI on JD, then UART 115200 on JD2
probe copies	JD7-JD10	UART TXD, UART RXD, I2C SCL, I2C SDA all the time

`COMx` is the PmodUSBUART's port (Device Manager on Windows, `/dev/ttyUSB*` on Linux). `pip install pyserial` first.

---

6. On the board: Way 2 (ARM + FPGA, Vitis app)

Open a terminal on the ZedBoard's own USB-UART (J14, 115200 8N1). After the app starts:

```
=== Smart Serial Controller v2 - Way 2 (ARM + FPGA) ===
controller v2 found at 0x43c00000
```

command	correct result	what it shows
`info`	ID 0x53534302, VERSION 0x00020000, JA=UART JB=SPI JC=I2C	the bitstream and register map
`id`	`JEDEC ID: 20 ba 19 (Micron - PmodSF3)`	v1 path, CPU drives the SPI engine
`temp`	`raw 0x0c 0x80 -> ... = +25.0000 C` and `CPU load of this one reading: about 13 APB accesses, 2 interrupts` (9 by the driver, 4 in the ISR)	v1 path and its CPU cost (slide 20 left)
`flowa`, then `pc_frame.py COMx 01 9F 03` on the PC	the PC gets `20 BA 19`; the console counts 1 packet each way	slide 17
`hub`	every 3.2 s: 32 records, seq increasing, t about 100000 us apart, src 3, len 2, `0c 80 = +25.0000 C`; `CPU load for 32 readings: 2 APB accesses (2 of them in the ISR), 1 interrupt(s)`	slides 18-20: the CPU sleeps, 64x fewer interrupts
`switch`	`JD while the SPI frame runs: still SPI`, `SW_CYCLES = ~30000 clocks (~300 us, it waited for the SPI frame)`, "HELLO FROM SM0 ON JD" on JD2, idle switch `SW_CYCLES = 2 clocks (20 ns)`	slide 14 and measurement 3
`se uart`	`HELLO FROM THE SERIAL ENGINE`, 0 errors, PASS	UART as a program
`se spi`	`READ ID -> 20 ba 19`	SPI as a program
`se i2c`	`TMP2 -> 0x0c 0x80 = +25.0000 C`	I2C as a program
`se man`	32 bytes, 0 errors, CRC equal, PASS	the new protocol (measurement 1)
`se oled HELLO`	the OLED shows "SSC V2 SERIAL ENGINE", "HELLO", the temperature	OLED driven by a program

The exact timing values on the board (baud rate, SCL frequency, switch time) are what measurement 2 and 3 of slide 24 record; see `docs/MEASUREMENTS.md`.